

BronchoTrack_V3 — Progress Report

Bronchoscope localization pipeline: architecture, development history, and current status

Prepared for: Thesis committee / supervising professor Prepared by: Aveesha Nishendra

Project: BronchoTrack_V3 — bronchoscope real-time
airway localization

Date: August 30, 2026

1. Executive Summary

BronchoTrack_V3 implements a real-time bronchoscope localization pipeline based on *BronchoTrack* (arXiv:2402.12763): a pre-operative 3-D airway graph is used to identify which airway branch the bronchoscope's camera is currently looking into, frame by frame, by detecting and tracking visible lumen (airway) openings and matching them against the graph's known branching structure. Two parallel implementations exist side by side in this codebase: **bronchotrack.paper_exact**, a strict reference build that reproduces the paper's method plus a small number of explicitly-flagged, opt-in extensions, and **bronchotrack.pipeline**, an earlier, more heavily embellished main package. **bronchotrack.paper_exact is the recommended and actively developed pipeline** as of this report.

This report covers the pipeline's full architecture, its exact per-frame algorithm (as pseudocode), and the development history of the current development cycle: a diagnosed failure mode in the baseline pipeline, an initial three-model "fusion" redesign that was built, tested, and ultimately set aside as a worse solution, and the smaller, targeted fix that replaced it and is now validated on real patient video. It closes with the current test and validation status and open items.

2. Pipeline Description

The pipeline processes a bronchoscopy video one frame at a time through four stages — detection, tracking, airway association (labeling), and localization — plus an optional appearance re-identification step feeding into tracking. Each stage is described below; Section 3 shows the same flow as a diagram, and Section 4 gives it as pseudocode.

2.1 Lumen Detection

A trained YOLO26 segmentation model (`detection.py`; the deployed checkpoint's own training metadata confirms `yolo26n-seg`) scans each video frame for visible lumen (airway-opening) candidates. Because the pipeline uses a segmentation checkpoint rather than a plain bounding-box detector, each raw model output carries: a bounding box (`x_c`, `y_c`, height, aspect ratio — the paper's own box parameterization), a confidence score, a class id, an image crop of the box region (for the re-identification step below), and — when a mask comes back for that detection — an (N, 2) polygon of the lumen's segmentation boundary, in pixel coordinates. The detector is deliberately run at a low confidence threshold (paper default: 0.1) so that low-confidence-but-real detections remain available to the tracker's second matching stage rather than being discarded outright.

2.2 Appearance Re-Identification (optional)

Each detection's image crop can optionally be passed through a ResNet50 feature extractor (`reid.py`) to produce a 2048-dimension appearance embedding. Each tracklet keeps a running exponential moving average of its own embedding ($e = 0.9 \cdot e_{\text{previous}} + 0.1 \cdot f_{\text{new}}$), giving the tracker an appearance cue (cosine similarity) it can use alongside motion when re-identifying a lumen across frames.

2.3 Multi-Lumen Tracking

A BYTE-style, two-stage tracker (`tracker.py`) maintains one constant-velocity Kalman filter per tracked lumen, over the state $[x_c, y_c, h, a, \dot{x}_c, \dot{y}_c, \dot{h}]$ (position, height, and aspect ratio, plus their velocities — aspect ratio itself is treated as constant, matching the paper). Each frame: (1) every existing tracklet is predicted forward one step; (2) high-confidence detections are matched against eligible tracklets using a combined cost $C = 0.5 \cdot C_{\text{appearance}} + 0.5 \cdot C_{\text{motion}}$ ($C_{\text{motion}} = 1 - \text{IoU}$, $C_{\text{appearance}} = 1 - \text{cosine similarity}$), via the Hungarian algorithm; (3) tracklets still unmatched are re-matched against low-confidence detections using motion cost only (appearance is not trusted for noisier, low-confidence boxes); (4) unmatched high-confidence detections spawn new tracklets, unmatched low-confidence detections are discarded; (5) tracklets unmatched for more than 30 consecutive frames are dropped. An eligibility hook lets the association stage below exclude tracklets whose current airway label is more than 3 generations away from the pipeline's current location estimate, per the paper's own filtering rule.

2.4 Airway Association (Labeling)

This is the pipeline's core algorithm (`paper_exact/association.py`): assigning each tracked lumen an anatomical branch label by matching it against the pre-operative 3-D airway graph.

- **Carina initialization.** The very first labels are assigned once exactly as many lumens are visible as the airway root (trachea) has children — typically the left/right main bronchus split — sorted left-to-right and matched to the graph's own left-to-right child ordering.
- **Label propagation.** From each already-labeled "anchor" tracklet, unlabeled tracklets nested inside its mask (children, i.e. openings visible past the anchor's own bifurcation) and unlabeled tracklets at the same level (siblings, under the anchor's own parent) are matched against the graph's known candidate branches via the Hungarian algorithm, on angular bearing from the anchor.
- **Roll-angle correction (paper Eq. 6/7).** The bronchoscope can rotate about its own optical axis; the pipeline estimates this roll from how the vector between the two oldest labeled tracklets rotates over time, Kalman-smooths that estimate, and rotates the graph's projected candidate positions by it before comparing bearings.
- **Diameter:distance verification cue (non-paper addition, on by default).** Beyond bearing, each candidate is scored on the ratio of its diameter to its distance from a reference point — to a small-angle approximation, its angular width as seen from that point. This is computed twice — once from the real detected mask, once from the 3-D graph at a "virtual viewpoint" a couple of centimetres past the bifurcation (approximating where the scope will actually be once a child branch reads clearly) — normalized within each domain to cancel the unknown pixel-to-millimetre scale factor, and blended into the matching cost. Each accepted match is additionally checked pass/fail against this cue independently (`Tracklet.diameter_distance_match`) — this is the flag that gates what actually gets drawn on the overlay video (Section 2.6).
- **Two-tier re-acquisition (new this cycle — see Section 5).** If every labeled anchor is lost at once, the pipeline no longer gives up permanently: it falls back to a frozen snapshot of the last known anchor to keep assigning labels. Full detail in Section 5.3.

2.5 Localization

Once tracklets carry labels, `localization.py` implements the paper's Eq. 8 voting rule every frame, with no temporal smoothing: each currently-visible labeled tracklet votes for its own branch if it is the only labeled lumen visible among its siblings, or for their shared parent branch if multiple sibling openings are visible at once (since which one the scope will advance into isn't yet resolved). The most-voted branch is reported as the bronchoscope's current location, together with its generation (0 = trachea, 1 = main bronchi, ...) read from the graph.

2.6 Output / Visualization

Three artifacts are produced per run: an overlay video drawing each confirmed lumen directly on the source footage, a schematic "graph view" video showing the scope's live position and full traversed path on the 3-D airway tree, and a per-frame JSON log. As of this cycle, the overlay draws each confirmed tracklet's actual segmentation mask — a translucent fill plus an outline — rather than a plain center marker, with its branch label above it, falling back to a center dot only when no mask is available for that tracklet. Display is deliberately gated on `diameter_distance_match is True` (Section 2.4), not merely "currently tracked": this lets the detector run at a permissive confidence threshold without cluttering the video, by leaning on the 3-D geometric verification to filter out the resulting false positives instead of a blunt per-frame confidence cutoff.

3. Pipeline Overview

The diagram below shows the per-frame data flow through the recommended (paper_exact) pipeline, from an incoming video frame to the three output artifacts, looping back for the next frame.

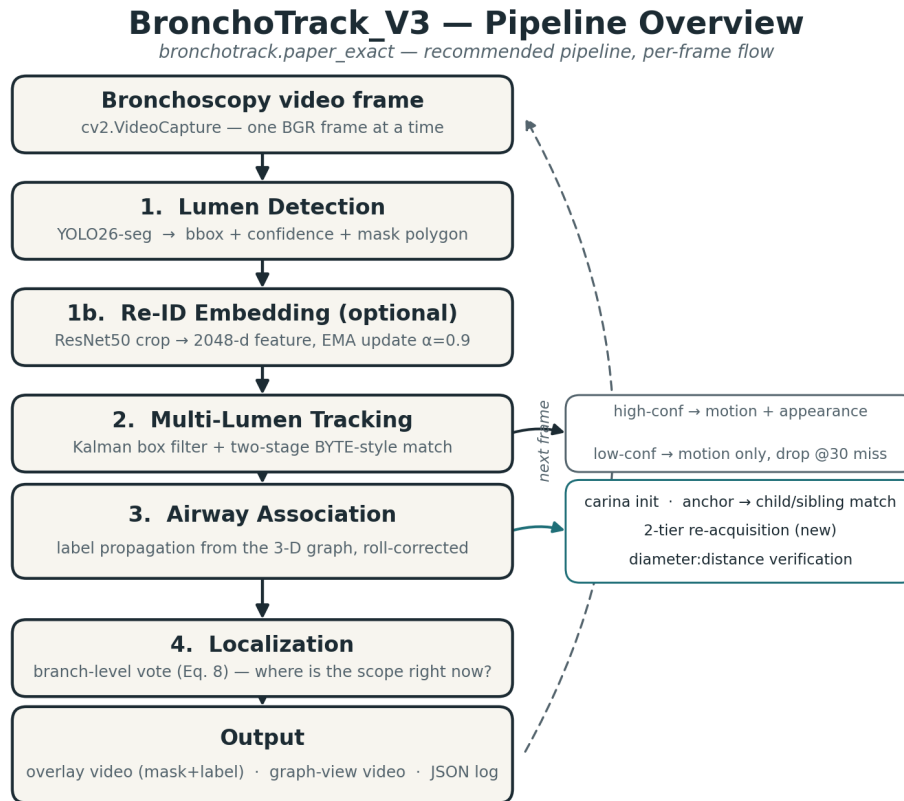


Figure 1. BronchoTrack_V3 per-frame pipeline flow (bronchotrack.paper_exact).

4. Pseudocode

The two blocks below give the pipeline's main per-frame loop and the airway-association algorithm that is this cycle's primary contribution (re-acquisition, marked NEW). Both are written directly from the corresponding source (paper_exact/pipeline.py and paper_exact/association.py), simplified for readability.

4.1 Main per-frame loop

```
function RUN_PIPELINE(video, airway_graph, detector_weights):
    graph      := AirwayGraph.load(airway_graph)
    detector    := LumenDetector(detector_weights)
    tracker     := MultiLumenTracker()
    assoc       := AirwayAssociation(graph)      # labeling + re-acquisition, Section 4.2
    localizer   := Localizer(graph)

    for frame, frame_idx in video.frames():

        # 1. Detect lumen openings
        detections := detector.infer(frame, frame_idx)      # bbox, confidence, mask

        # 1b. (optional) appearance embedding for re-identification
        for d in detections:
            d.embedding := reid_embedder.embed(d.crop)

        # 2. Track: predict + two-stage BYTE-style match (Section 2.3)
        tracklets := tracker.update(detections, frame_idx,
                                    eligibility_fn = assoc.eligibility_fn)

        # 3. Associate: propagate anatomical labels from the airway graph
        labels := assoc.process_frame(tracklets, frame_idx)

        # 4. Localize: branch-level vote (Eq. 8)
        location := localizer.localize(tracklets)
        generation := graph.generation(location)

        result := FrameResult(frame_idx, detections, tracklets,
                               labels, location, generation)

        write_overlay_frame(result)      # mask + label overlay
        write_graph_view_frame(result)    # scope position on 3-D tree
        append_json_log(result)

    return all_results
```

4.2 Airway association — label propagation and re-acquisition

```
function ASSOCIATION.PROCESS_FRAME(tracklets, frame_idx):
    current := [t in tracklets : t.time_since_update == 0]    # visible this frame

    if not initialized:
        TRY_INITIALIZE_CARINA_SPLIT(current, frame_idx)        # Section 2.4
        if still not initialized:
            return {}

    UPDATE_ROLL_ESTIMATE(current)    # Kalman-filtered, Eq. 6/7

    anchors := [t in current : t.label is not None]
    sort anchors by age, oldest/most-established first
    if anchors is not empty:
```

```

    # remember this frame's best anchor as a frozen snapshot, in
    # case every real anchor is lost on some later frame
    last_real_anchor      := freeze(anchors[0])
    last_real_anchor_frame := frame_idx
else:
    anchors := REACQUIRE(current, frame_idx)          # NEW

used_labels := { a.label for a in anchors }
unlabeled   := [t in current : t.label is None]

for anchor in anchors:
    unlabeled := PROPAGATE_FROM_ANCHOR(anchor, unlabeled,
                                       used_labels, frame_idx)

update_gallery(current, frame_idx)    # record every branch ever visited
return { t.track_id: t.label for t in current if t.label is not None }

function REACQUIRE(current, frame_idx):                # NEW
    # Called only when every real anchor was lost this frame.
    if reacquisition disabled, or no snapshot ever taken, or
    frame_idx - last_real_anchor_frame > max_gap_frames:
        return []                                     # give up (for now)

    snapshot := last_real_anchor
    unlabeled := [t in current : t.label is None]

    # Tier 1 -- SELF re-acquisition: the scope's own current lumen
    # dropped out for a few frames and came back, e.g. motion blur.
    if unlabeled is not empty:
        best := argmax_{t in unlabeled} IoU(snapshot.last_box, t.last_box)
        if IoU(snapshot.last_box, best.last_box) >= iou_threshold:
            best.label := snapshot.label
            return [best]                                # a real anchor again, immediately

    # Tier 2 -- VIRTUAL-ANCHOR fallback: nothing still resembles the
    # old position (the scope kept moving) -- hand the frozen snapshot
    # itself back in as a stand-in anchor so genuinely NEW child /
    # sibling lumens can still be matched against where it last was.
    return [snapshot]

function PROPAGATE_FROM_ANCHOR(anchor, unlabeled, used_labels, frame_idx):
    children := [t in unlabeled : IS_NESTED(anchor, t)]    # inside anchor's mask
    match CHILDREN(anchor.label) <-> children
    via Hungarian assignment on:
        cost = (1 - w)*bearing_cost + w*diameter_distance_cost

    siblings := [t in unlabeled : not nested either direction]
    match SIBLINGS(parent(anchor.label)) <-> siblings    (same cost formula,
                                                         projected from the parent)

    return unlabeled minus every tracklet just matched

```

5. Development History This Cycle

This section covers the full progress from a diagnosed failure in the working baseline through to the current recommended pipeline, including a redesign that was built, evaluated, and then deliberately set aside in favor of a smaller, targeted fix.

5.1 Starting point: a diagnosed anchor-loss dead end

The baseline `paper_exact` pipeline (Section 2) was already working well, but label propagation (Section 2.4) only ever runs outward from a currently-visible, already-labeled tracklet (an "anchor"). Read literally, this has a real dead end: if every anchor is lost at the same time — a long occlusion, or a burst of missed detections — nothing in the pipeline can ever assign a new label again for the rest of the video. Log analysis of a real patient video confirmed this in practice: the pipeline reported zero locations for 82% of a 1084-frame run, after its last anchor was lost around frame 200.

5.2 First attempt: a three-model fusion redesign (built, evaluated, set aside)

The first approach taken was a larger redesign, built from scratch as a new `bronchotrack.fusion` package rather than a patch to `paper_exact`. It combined three separate models every frame instead of relying on anchor persistence at all:

- **An approach motion model** — a scale-free "time-to-contact" estimate ($\tau = \text{diameter} / \text{rate-of-change-of-diameter}$) derived purely from how fast a tracked lumen's apparent size is growing, with no camera calibration required, cross-checked against the graph's own known branch lengths.
- **A point-based identification cue** ("the bronchotrack version") — the paper's own angular-bearing idea, re-derived independently using each lumen's segmentation center.
- **A whole-mask "ratio method" identification cue** — using the entire detected mask boundary (area-equivalent diameter, boundary-to-boundary gap) rather than reducing each lumen to a single point, to make deliberate use of the segmentation output rather than a plain bounding box.
- **A Kalman fusion layer** combining all three into one persistent, per-branch confidence score every frame — with no one-shot initialization gate, so losing every tracklet only ever caused a few frames of prediction-only coasting rather than a permanent failure.

This was implemented in full (motion model, both identification cues, Kalman fusion, candidate-branch overlay display), covered by 18 hand-written regression tests, and run end-to-end on real patient video. It was then reviewed against the working baseline and **rejected as a net regression** in favor of reverting to the `paper_exact` baseline and applying a smaller, targeted fix instead (Section 5.3). The fusion package remains in the codebase, still passing its own tests, but is **not** the recommended or actively developed pipeline going forward.

5.3 Adopted fix: two-tier re-acquisition in `paper_exact`

Rather than removing anchor-dependence altogether, the adopted fix keeps the baseline's architecture and adds a bounded fallback for the specific failure mode diagnosed in 5.1. `AirwayAssociation` now keeps a frozen snapshot of the most-established real anchor, refreshed every frame one exists. The moment a frame has no real anchor, re-acquisition tries two things in order, both bounded by a maximum staleness window (default 90 frames, ≈ 3 seconds at 30 fps, past which the snapshot is considered too stale to trust):

- **Tier 1 — self re-acquisition.** The dominant real-world case: the lumen the scope is already inside drops out for a few frames (motion blur, a burst of missed detections) and reappears as itself, not as a newly-visible child. Among this frame's unlabeled tracklets, whichever one overlaps the snapshot's last known box the most (by IoU) is relabeled directly with the snapshot's own label if that overlap clears a threshold (default 0.3) — no Hungarian matching needed, since the question is "is this probably the same lumen", not "which child is this".
- **Tier 2 — virtual-anchor fallback.** If nothing in the current frame still resembles the snapshot's old position (the scope kept advancing while the anchor was lost), the frozen snapshot itself is handed to the normal label-propagation step exactly as a live anchor would be, so genuinely new child or sibling lumens can still be matched against where the anchor last was.

Either tier immediately restores a real anchor from the very next frame onward — this is a per-frame fallback, not a second one-shot initialization, so it keeps working across repeated gaps for the rest of a video. Setting the staleness window to 0 disables the feature entirely and reproduces the original dead-end behavior, for direct before/after comparison.

5.4 Latest change: full-mask overlay visualization

Most recently, the overlay video (Section 2.6) was changed to draw each confirmed lumen's actual segmentation polygon — a translucent fill plus outline — instead of a fixed-radius center dot, with a fallback to the original dot marker only when no mask is available for a given tracklet. This is a visualization change only; it does not affect labeling or localization logic.

6. Quantitative Results on Real Patient Video

The re-acquisition fix was validated end-to-end on the same real, 1084-frame patient video used to originally diagnose the anchor-loss dead end (Section 5.1), comparing a run with re-acquisition disabled against one with it enabled at default settings, using identical detector weights, confidence threshold, and airway graph. (Processing-speed / frame-rate figures are omitted here as not relevant to localization accuracy.)

Metric	Before (re-acquisition disabled)	After (re-acquisition enabled)	Change
Frames with a labeled location	166 / 1084 (15.3%)	433 / 1084 (39.9%)	$\approx 2.6\times$
Frames with a 3-D-model-confirmed dot	162 / 1084 (14.9%)	156 / 1084 (14.4%)	essentially flat
Final reported location	—	L231 (generation 4)	

Table 1. Before/after comparison, same 1084-frame video, same methodology.

Re-acquisition delivers its intended effect directly: the pipeline can now assign a location on $2.6\times$ as many frames, confirming it no longer freezes permanently after the anchor-loss point that produced the original 82%-blank result. The second row is a genuine, and important, nuance rather than an inconsistency: the "confirmed dot" count is governed by a separate, stricter gate (the diameter:distance virtual-verification check, Section 2.4/2.6) that re-acquisition does not directly improve — a re-acquired label can be correct while still not clearing that independent 3-D geometric agreement check on a given frame. Visual spot-checks of the overlay video confirm this reading: early frames near the carina show both branches correctly labeled and confirmed, while later frames (e.g. frame 700, deep in the tree at L231) show the correct location text but no confirmed dot, consistent with the flat second metric.

The location distribution across the full re-acquisition run was dominated by L231 (881 of 1084 frames), with L23 (91), trachea (78), L2 (18), and R (15) making up most of the remainder — consistent with a single, mostly-monotonic advance down the left bronchial tree to generation 4.

7. Testing and Validation

All changes this cycle are covered by an expanded automated test suite, run both in place and from a freshly extracted copy of the delivered codebase before each hand-off:

Suite	Covers	Result
test_pipeline_smoke.py	Main package (bronchotrack.pipeline)	17 / 17
test_paper_exact_smoke.py	Recommended pipeline, incl. re-acquisition (4 new tests) and mask-overlay rendering (1 new test)	23 / 23
test_fusion_smoke.py	Fusion package (retained, not recommended)	18 / 18

Table 2. Automated test results, 58 / 58 total, verified from a clean zip extraction.

New re-acquisition tests specifically cover: correct relabeling of the same lumen after a total, genuine tracker drop; re-acquisition correctly giving up once the staleness window is exceeded; the disable escape hatch (window = 0) reproducing the original behavior exactly; and tier-2 virtual-anchor propagation correctly matching a new child when tier-1 self-matching does not apply. The new mask-overlay test confirms the full polygon is actually rendered (not just a marker) and that its fill is translucent rather than opaque.

8. Current Status and Open Items

- **Recommended pipeline:** `bronchotrack.paper_exact`, with re-acquisition enabled at default settings and the new mask-overlay visualization.
- **Not recommended, retained in codebase:** `bronchotrack.fusion` (Section 5.2) — fully implemented and tested, but not the active development path.
- **Open item — raw mask artifacts at the frame border.** Now that the overlay draws full segmentation polygons rather than a single point, a pre-existing quirk in the raw detector output is newly visible: a lumen mask that gets clipped at the edge of the video frame can include polygon points that trace along the image border, rendering as an oversized shape reaching toward the frame's corners. This is not a labeling or localization defect — it is a property of the raw segmentation output becoming visible now that the full mask is drawn — but may be worth a border-touching sanity filter if it proves distracting in the overlay video.
- **Open item — confirmed-dot coverage.** The 3-D-model-verification gate that controls whether a label is actually displayed remained flat under re-acquisition (Section 6). Whether this is acceptable as-is, or whether the virtual-verification threshold should be revisited for re-acquired (as opposed to freshly Hungarian-matched) labels specifically, is an open design question for discussion.

Deliverables accompanying this report: the full updated codebase (`bronchotrack_pipeline.zip`), and the re-acquisition + mask-overlay validation run on real patient video (overlay video, graph-view video, per-frame JSON log).